《基于静态与动态语义分析的 OpenHarmony 程序修复工具应用研究》开题报告

1. 研究意义

当前，OpenHarmony 应用在复杂场景下的崩溃率仍高于部分主流系统，但现有的自动化程序修复（APR）工具对 ArkTS 语言及声明式范式的支持存在显著空白。在实际工程中（如本研究选用的鸿蒙 SpringBoot 多模态翻译软件），广泛应用了 TaskPool 异步多线程（用于处理 OCR 和语音翻译）、网络并发请求及复杂的 ArkUI 组件生命周期管理。这类高并发与声明式状态绑定极易引发空指针、资源未释放或竞态条件等隐蔽缺陷，纯人工排查与修复的成本极高。

本研究的意义在于填补 OpenHarmony 生态内工具链级应用的空白，核心聚焦于集成现有成熟分析工具，而非从零自研底层算法。通过打通 ArkAnalyzer 的静态数据流与 HapTest 的动态轨迹，并将其适配至 SemFix 补丁生成引擎，构建首个针对 ArkTS 的自动化修复流水线。这不仅能有效提升 OpenHarmony 原生应用的缺陷治理效率，也为产学研工具的落地化集成提供了极具参考价值的工程实践范式。

2. 相关现状调研

当前，针对 Java、C/C++等传统语言的自动化修复工具链已趋于成熟，但面向 OpenHarmony ArkTS 声明式 UI 编程范式的研究仍处于起步阶段。现有技术虽有初步成果，但呈现出显著的“数据孤岛”现象与技术断层痛点：

• ArkAnalyzer：作为官方支持的静态分析框架，长于 ArkTS 的抽象语法树(AST)解析与控制流图(CFG)构建，但止步于缺陷检测，无法提供修复建议。

• HapTest：作为优秀的运行时异常监控框架，可通过字节码插桩捕获动态 Trace 轨迹，却面临“异常轨迹”到“源代码具体行号”的映射断层，无法为修复引擎提供精准锚点。

• SemFix：具备强大的 SMT 语义约束求解与补丁生成能力，但其输入目前严重依赖人工提供缺陷定位和约束条件，且原生不支持 ArkUI 的声明式语法。

缺少统一的中间表示（IR）层来打通“静态检测-动态验证-约束求解”的数据流转，是当前最大的技术瓶颈。因此，打破工具孤岛、构建全流程自动化修复工具链成为了本课题的核心定位。

3. 所选择工具初步搭建情况

本研究的核心思路是将三款成熟工具串联成完整的修复流水线，作用于鸿蒙前端及SpringBoot 架构。目前各工具的初步情况与能力如下：

ArkAnalyzer（缺陷检测模块）：作为 OpenHarmony 官方与华为开发的 ArkTS 静态分析框架，目前已发布 v1.0 版本。其关键能力包括 ArkTS 语法树(AST)解析、语义分析（类型检查、作用域解析）、数据流分析以及控制流分析，可用于检测类型安全、资源管理及生命周期等方面的缺陷。

HapTest（异常监控模块）：该运行时动态分析框架主要监控内存泄漏和异常捕获，通过字节码插桩和运行时 Hook 技术，能够输出包含时间戳、调用栈和变量值的执行轨迹 Trace 文件 。

SemFix（补丁生成模块）：利用语义约束生成修复方案，通过将程序行为转化为逻辑公式并使用 SMT 求解器，生成能够替换缺陷位置的候选表达式。

初步搭建的验证场景为开源的 Translation 应用（地址：https://gitcode.com/qq_74195490/OpenHarmony ），集成了文字、拍照、语音翻译及用户注册登录功能。目前已初步分析出应用存在的实际问题，如 API 密钥硬编码、空值处理不完善以及生命周期资源泄漏等。集成方式计划采用开发 Python/TypeScript 胶水代码，通过文件或 API 进行数据交换。

4. 进度安排与预期成效

实施计划与时间安排：

第 1-2 周：环境搭建与工具熟悉。安装 DevEco Studio 5.0+，配置 OpenHarmony SDK，部署 ArkAnalyzer 和 HapTest 框架

第 3-4 周：静态分析数据提取。使用 ArkAnalyzer 扫描应用，收集并分类缺陷报告，构建缺陷位置索引表

第 5-6 周：动态监控数据采集。配置 HapTest 监控规则，设计覆盖主要功能的测试用例并收集执行轨迹

第 7-9 周：数据对接块开发。实现静态到动态的数据关联算法，以及动态到修复约束的转换模块

第 10-12 周：修复生成与验证。适配 SemFix 到 ArkTS 语法，实现补丁自动验证脚本并在应用上测试

第 13-16 周：总结与文档撰写，整理工具使用指南及答辩材料

预期成效：

软件成果：产出实现三款工具自动串联的胶水代码、统一的缺陷描述数据格式规范，以及提供一键修复功能的 DevEco Studio 插件原型。

数据集成果：形成包含 50+真实缺陷样本的缺陷基准集，以及覆盖主要异常场景的自动化测试用例库

文档成果：撰写《OpenHarmony 程序修复工具应用指南》及面向开发者的工具链使用教程

量化指标：预计本集成方案的缺陷检出率将达到 92%，修复成功率达到 67%，并将误报率控制在 10%

5. 参考文献

ArkAnalyzer 理论基础：参考《OpenHarmony 技术报告 2024》及发表于 ICSE-SEIP (2025) 的相关论文

HapTest 理论基础：参考《OpenHarmony 技术报告 2024》

SemFix 理论基础：参考发表于 ICSE 2013 的核心论文